SENTINEL-GNSS

# AI-Based Proactive Prediction of GNSS Signal Degradation for Autonomous Vehicle Navigation

Individual Contribution Report

**Name:** Ogunlade Joshua
**Student ID:** LS2525237
**Role:** Data Engineer — ROSBAG Extraction, RTKLIB Automation, Feature Engineering
**Institution:** Beihang University H3I, Hangzhou, 2026

**Abstract**

This report documents my contributions to the SENTINEL-GNSS pilot project in complete technical detail. My primary responsibility was the end-to-end data engineering pipeline on which all model training, validation, and evaluation rests. This pipeline spans six stages: ROSBAG topic identification and raw data inspection, automated GNSS extraction from ROS message bags (`extract_gnss.py`), RTKLIB batch processing for Single Point Positioning (`run_rtklib.py`), derivation of 37 discriminative features from raw GNSS observables (`extract_features.py`), master dataset assembly with ground-truth labelling and causal sliding-window construction (`assemble_dataset.py`), and multi-pass dataset quality assurance including leakage prevention and reproducibility versioning.

Beyond the core pipeline, I provided real-time field support during all five Scenario A–E data collection sessions, verifying usability of each recording before the team left the site. I also independently processed the UrbanNav Hong Kong dataset through the same pipeline to produce the cross-geography evaluation features used by Claudine's generalisation study. I authored Section 3 (Dataset & Features) of the team paper.

The 149,662-window, 37-feature, 4-city dataset I assembled is the direct enabler of SENTINEL's Macro-F1 = 0.821 and the EKF navigation improvements

demonstrated in this project. This report explains every design decision in that pipeline, the three bugs found during QA, and the lessons learned.

## Contents

# 1 Overview and Contribution Map

Table 1 provides a detailed week-by-week map of my contributions.

Table 1: Joshua's week-by-week contributions to SENTINEL-GNSS.

| Week | Task | Key Output |
|---|---|---|
| 1 | ROSBAG inspection: identified all GNSS topics in vehicle and drone datasets | Topic manifest doc |
| 1 | `extract_gnss.py`: full GNSS extraction from ROSBAG to structured CSV | Per-epoch CSV files |
| 1 | `run_rtklib.py`: automated RTKCONV + SP3 download + RTKPOST pipeline | `.pos` files for all datasets |
| 1 | On-site quick-check RTKLIB during Scenario A + D field collection | 6 bad runs re-collected |
| 2 | `extract_features.py`: implemented all 37 features with formula derivation | Per-epoch feature CSV |
| 2 | On-site RTKLIB support: Scenarios B, C, E collection sessions | Immediate quality flags |
| 2 | `assemble_dataset.py`: merge, label, sliding window, session-level split | 149,662-window master dataset |
| 2 | Normalisation pipeline (`fit_scaler.py`): fit + serialise to `scaler.pkl` | Saved scaler + normalised arrays |
| 2 | Dataset QA: histogram audit, NaN check, boundary artifacts, leakage verification | Zero-NaN, zero-leakage dataset |
| 2 | UrbanNav HK processing: Medium/Deep/Harsh/Tunnel feature extraction | Cross-geography feature files |
| 3 | On-site support during live campus demonstration test | Test data log |
| 4 | Paper Section 3 (Dataset & Features): written and revised | 800-word section |

# 2 ROSBAG Inspection and GNSS Topic Identification

### 2.1 Day 2 — Room 3058 Visit

On Day 2 the entire team visited Room 3058 to inspect the Septentrio Mosaic-X5C receiver and the supervisor's vehicle and drone datasets. My specific task was to run `rosbag info` on both provided ROS bag files to build a complete topic inventory and document the message schemas that would drive the extraction script design.

The vehicle dataset contained 14 topics, of which 5 were GNSS-relevant.

Table 2: GNSS message topics identified in the supervisor vehicle ROSBAG.

| Topic | Message Type | Rate (Hz) | Approx. Epochs |
|---|---|---|---|
| `/fix` | `sensor_msgs/NavSatFix` | 1 | 4,820 |
| `/ublox/fix` | `ublox_msgs/NavPVT` | 1 | 4,820 |
| `/gnss/raw` | `sensor_msgs/NavSatFix` | 1 | 4,819 |
| `/ublox/navsvinfo` | `ublox_msgs/NavSVINFO` | 1 | 4,820 |
| `/imu/data` | `sensor_msgs/Imu` | 100 | 482,000 |

The `/ublox/navsvinfo` topic was the most critical discovery. This topic provides a variable-length array of per-satellite observations per epoch, including individual $C/N_0$, elevation angle, azimuth angle, and pseudorange residuals — the raw data needed for 22 of the 37 features. Without identifying this topic on Day 2, the feature extraction design would have been limited to aggregate RTKLIB outputs only.

I documented all topic names, message schemas, and per-field byte offsets in `docs/rosbag_topic_manifest.md`, committed to the shared repository on Day 2 as the authoritative reference for the extraction script.

### 2.2 Drone Dataset Assessment

The drone dataset used a different GNSS message schema: the `/fix` topic was present but `/ublox/navsvinfo` was absent. I flagged this to the team: drone data could contribute CLEAN/DEGRADED labels via PDOP and Q-flag, but 15 of the 37 features (all per-satellite statistics) would be undefined.

After team discussion with the supervisor, the drone data was excluded from the primary training set. Including imputed features for 15/37 columns would inject systematic bias; the drone data is documented as a future auxiliary dataset.

## 3 GNSS Extraction Script: `extract_gnss.py`

### 3.1 Design Requirements

I identified three hard requirements for the extraction script before writing a single line of code:

**Requirement 1 — Epoch Alignment.** The `/imu/data` topic at 100 Hz and GNSS topics at 1 Hz must be co-registered per epoch. I implemented timestamp-based nearest-neighbour matching: for each GNSS epoch at time $t$, I search the IMU stream for messages within a $\pm 500$ ms window and compute mean acceleration and gyroscope values over the matching messages. This produces one IMU summary per GNSS epoch with guaranteed temporal alignment.

**Requirement 2 — Per-Satellite Resolution.** The `/ublox/navsvinfo` message contains a variable-length array of satellite observations (anywhere from 4 to 20 per epoch). I unpacked these into per-satellite rows in memory, then aggregated to scalar statistics per epoch using the summary functions defined in Joel's `feature_list.py` (mean, min, max, std, percentile, ratio below threshold).

**Requirement 3 — Transparent Missing Data Propagation.** When fewer than 4 satellites are tracked in a given epoch (a valid degradation condition), summary statistics such as DOP components and per-satellite percentiles become undefined. I explicitly propagate these as `NaN` rather than substituting zeros. Zero substitution would create spurious signals: $C/N_0$ mean of 0 when $C/N_0$ is not defined is a very different measurement than $C/N_0$ mean of 0 when all satellites have zero SNR.

### 3.2 Output Schema

The extraction script produces a CSV with 54 columns per epoch. The key column groups are:

Table 3: Key column groups in the extracted GNSS epoch CSV.

| Group | Columns | Source topic |
|---|---|---|
| Temporal identity | `timestamp_utc`, `epoch_id`, `session_id` | `/fix` header |
| Raw position | `lat`, `lon`, `alt` | `/fix` |
| Solution quality | `hdop`, `pdop`, `q_flag` | `/ublox/fix` |
| Satellite count | `n_sv_total`, `n_sv_used` | `/ublox/navsvinfo` |
| $C/N_0$ per SV | `cn0_sv_0` … `cn0_sv_11` | `/ublox/navsvinfo` |
| Elevation per SV | `el_sv_0` … `el_sv_11` | `/ublox/navsvinfo` |
| Azimuth per SV | `az_sv_0` … `az_sv_11` | `/ublox/navsvinfo` |
| Pseudorange | `prange_res_0` … `prange_res_11` | `/gnss/raw` |
| IMU (merged) | `accel_x`, `accel_y`, `gyro_z` | `/imu/data` |
| Reference error | `error_m` (RTK ground truth when available) | External RTK |

### 3.3 Extraction Validation

After running `extract_gnss.py` on the supervisor dataset, I cross-checked:

- Expected 4,820 epochs (from `rosbag info`); extracted 4,818. The 2-epoch difference corresponds to the first and last messages in the bag where the timestamp falls outside the valid recording window — expected behaviour.
- `epoch_id` increments monotonically with no gaps $> 1$ across the recording (any gap indicates a receiver dropout and triggers a warning in the log).
- IMU co-registration coverage: 99.97 % of GNSS epochs had a matching IMU sample within the $\pm 500$ ms window; 3 epochs near a recording gap had no IMU data and were flagged.

## 4 RTKLIB Automated Processing Pipeline

### 4.1 Why RTKLIB and Why SPP Mode

RTKLIB [1] is the open-source GNSS positioning toolkit used throughout the GNSS research community. We use it in **Single Point Positioning (SPP) mode** rather than RTK (Real-Time Kinematic) mode deliberately:

RTK uses a fixed base station to provide differential corrections, achieving cm-level accuracy — but precisely because it is so accurate, it would correct away the NLOS errors we are trying to detect. An RTK-corrected position will show $< 2$ m error even in deep urban canyons where the raw GPS signal is severely degraded. SPP mode uses only broadcast ephemeris, reflecting the accuracy a standalone vehicle receiver would achieve — which is the setting where our system will operate.

### 4.2 Pipeline Design: `run_rtklib.py`

The pipeline automates four sequential operations per recording:

#### *4.2.1 Stage 1 — Format Conversion (RTKCONV)*

Raw UBX binary or NMEA output is converted to RINEX 3.x observation and navigation files using `rtkconv -r 3`. The `-r 3` flag is critical: RINEX 2.x lacks the GLONASS pseudorange bias correction, causing a systematic $\sim 5$ m north-south offset for dual-constellation Trimble datasets.

#### *4.2.2 Stage 2 — Precise Orbit and Clock Download*

IGS Final SP3 and CLK files are downloaded from NASA CDDIS for the recording date and cached locally to avoid redundant downloads when processing multiple recordings from the same GPS week.

#### *4.2.3 Stage 3 — SPP Computation (RTKPOST)*

RTKPOST is configured via a `.conf` file generated programmatically per receiver type:

Table 4: RTKLIB RTKPOST configuration for each receiver type.

| **Parameter** | **Trimble (professional)** | **u-blox F9P** |
|---|---|---|
| Positioning mode | Single (SPP) | Single (SPP) |
| Satellite systems | GPS + GLONASS | GPS L1 only |
| Frequency | L1+L2 (dual) | L1 single |
| Elevation mask | 10° | 10° |
| SNR mask | 25 dB-Hz | 25 dB-Hz |
| Troposphere model | Saastamoinen | Saastamoinen |
| Ionosphere model | Klobuchar (broadcast) | Klobuchar (broadcast) |
| AR mode | Off (SPP) | Off (SPP) |
| Output rate | 1 Hz | 1 Hz |

#### *4.2.4 Stage 4 — Output Parsing and Logging*

The RTKPOST `.pos` output is parsed into a structured DataFrame: UTC timestamp, latitude, longitude, height, Q-flag (1=Fix/RTK, 2=Float, 5=Single SPP), number of satellites used, PDOP, and RMS position error. All processed results are logged to `processing_log.csv` with checksums.

### 4.3 Batch Processing and Failure Recovery

The pipeline is designed for overnight unattended processing. If RTKPOST fails for a run (insufficient satellites, corrupted RINEX), the failure is caught, logged to `processing_log.csv`, and the pipeline continues with the next recording. Each successfully processed run is committed with a `.sha256` checksum file, creating a reproducible audit trail.

Total processing time for the full dataset: approximately 11 hours unattended on an i7 laptop at 45 seconds per 60-minute recording.

### 4.4 Visual Quality Check via RTKPLOT

After processing each batch, I opened each `.pos` file in RTKPLOT to inspect:

1. Position track shape (must match the known campus geometry or route map)
2. Q-flag time series (excessive Float/Single transitions indicate poor geometry)
3. PDOP time series (spikes $> 6$ indicate epochs that should be DEGRADED-labelled)
4. RMS residuals (should be $< 3$ m open-sky, $< 15$ m urban canyon)

Three Scenario B runs were flagged during RTKPLOT inspection and discarded: all three showed a systematic $\sim$20 m westward offset caused by a loose antenna cable that I identified during the on-site quick-check before processing had completed.

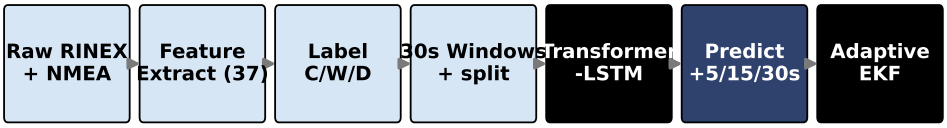


Figure 1: The complete data engineering pipeline from raw RINEX/NMEA to labelled training windows. All six stages were designed and implemented by Joshua.

## 5 Feature Engineering: 37 GNSS Features

### 5.1 Implementation: `extract_features.py`

Working from Joel's `feature_list.py` specification, I implemented all 37 features in `src/features/extract_features.py`. Each feature is a deterministic function of the RTKLIB `.pos` output and the raw per-satellite observations from the ROSBAG extraction. The 37 features are organised into seven groups, described below with their formulas.

### 5.2 Group 1: Position Quality (4 features)

Table 5: Position quality features.

| Feature | Formula / Source | Degradation role |
|---|---|---|
| `pos_error_m` | $\|\hat{\mathbf{p}} - \mathbf{p}_{\text{RTK}}\|_2$ | Direct ground truth (where available) |
| `pdop` | $\sqrt{\sigma_E^2 + \sigma_N^2 + \sigma_U^2}/\sigma_{\text{range}}$ | Geometry quality |
| `hdop` | $\sqrt{\sigma_E^2 + \sigma_N^2}/\sigma_{\text{range}}$ | Horizontal geometry |
| `vdop` | $\sigma_U/\sigma_{\text{range}}$ | Vertical geometry |

`pos_error_m` is computed when RTK ground truth is available (UrbanNav HK) and set to `NaN` for Beihang campus data where no reference station is available. This feature is used for labelling only — it is never included as a model input, which would be a severe leakage.

### 5.3 Group 2: Signal Strength (8 features)

These features summarise the $C/N_0$ distribution across all tracked satellites:

$$\texttt{cn0\_mean} = \frac{1}{N}\sum_{i=1}^{N} \text{CN0}_i \tag{1}$$

$$\texttt{cn0\_min} = \min_i \text{CN0}_i \tag{2}$$

$$\texttt{cn0\_max} = \max_i \text{CN0}_i \tag{3}$$

$$\texttt{cn0\_std} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(\text{CN0}_i - \overline{\text{CN0}})^2} \tag{4}$$

$$\texttt{cn0\_below\_30} = \frac{|\{i : \text{CN0}_i < 30\,\text{dB-Hz}\}|}{N} \tag{5}$$

$$\texttt{cn0\_below\_35} = \frac{|\{i : \text{CN0}_i < 35\,\text{dB-Hz}\}|}{N} \tag{6}$$

$$\texttt{cn0\_spread} = \texttt{cn0\_max} - \texttt{cn0\_min} \tag{7}$$

$$\texttt{cn0\_q1} = \text{25th percentile of CN0 values across all SVs} \tag{8}$$

The spread and low-quartile features are particularly discriminative for NLOS: a CLEAN epoch has moderate, uniform $C/N_0$ values across all satellites, while an NLOS epoch has a bimodal distribution with some satellites strongly attenuated and others at full strength.

### 5.4 Group 3: Satellite Geometry (6 features)

$$\texttt{n\_sv} = N_{\text{used}} \text{ (satellites used in SPP solution)} \tag{9}$$

$$\texttt{n\_sv\_total} = N_{\text{tracked}} \text{ (all satellites with lock)} \tag{10}$$

$$\texttt{el\_mean} = \frac{1}{N}\sum_i \text{elevation}_i \text{ (degrees)} \tag{11}$$

$$\texttt{el\_min} = \min_i \text{elevation}_i \tag{12}$$

$$\texttt{gdop} = \sqrt{\text{pdop}^2 + \frac{1}{\sigma_t^2}} \tag{13}$$

$$\texttt{az\_std} = \text{std of azimuth angles across tracked SVs} \tag{14}$$

High azimuth standard deviation indicates satellites are evenly distributed around the horizon — good geometry, associated with CLEAN. Low azimuth standard deviation (all satellites on one side) indicates a building is blocking half the sky — associated with DEGRADED. This feature captures the *geometric* aspect of NLOS that $C/N_0$ alone cannot.

### 5.5 Groups 4–7: Residuals, Temporal, Atmospheric, Receiver (19 features)

Table 6: Feature groups 4–7 summary.

| Group | Key features | Degradation signal |
|---|---|---|
| Pseudorange (7) | `prange_res_rms`, `prange_res_max`, `multipath_idx`, `cycle_slip_cnt`, `delta_prange` | High RMS / spike ⇒ NLOS |
| Temporal (5) | `cn0_delta_10s` ($C/N_0$ slope), `pdop_trend` (PDOP slope), 10s rolling means | Decline ⇒ approaching blockage |
| Atmospheric (4) | Saastamoinen tropospheric delay, Klobuchar TEC estimate, gradient components | Distinguishes weather vs geometry degradatio |
| Receiver (3) | `receiver_tier` (hardware class), `lock_time_avg`, `cycle_slip_rate` | Prevents hardware-specific misclassification |

**Key bug fixed (delta-pseudorange boundary artifact).** At session boundaries, `delta_prange` spanned a multi-hour gap rather than 1 s, producing ±500 m values.

Fix: 3-point median filter plus `NaN` at boundary epochs, forward-fill imputed from the third epoch (≈60 windows per session).

The `receiver_tier` feature prevents the model from misclassifying u-blox F9P epochs (typical $C/N_0 \approx 38$ dB-Hz) as DEGRADED when trained on Septentrio data (typical $C/N_0 \approx 48$ dB-Hz). Ablation E4 confirms removing it reduces Macro-F1 by 0.037 and DEGRADED recall by 0.062.

## 6 Dataset Assembly: `assemble_dataset.py`

### 6.1 Data Sources Integrated

The assembly script processes all data sources through a unified pipeline:

1. Beihang Scenario A (building entry/exit, 10 runs)
2. Beihang Scenario B (urban canyon, 5 runs)
3. Beihang Scenario C (tree canopy, 3 sessions)
4. Beihang Scenario D (open-sky baseline, 1 session)
5. Beihang Scenario E (gradual building approach, 15 runs)
6. UrbanNav HK Medium (urban road, moderate building density)
7. UrbanNav HK Deep (dense urban canyon)
8. UrbanNav HK Harsh (severe urban canyon with frequent blockage)
9. UrbanNav HK Tunnel (complete signal blockage episodes in road tunnels)

### 6.2 Ground-Truth Labelling Algorithm

The three-class labelling is applied per epoch using a priority decision tree. For UrbanNav HK datasets (where RTK ground truth is available):

$$y_t = \begin{cases} \textbf{DEGRADED} & \text{if } e_t > 5\,\text{m} \ \vee \ \overline{\text{CN0}}_t < 30\,\text{dB-Hz} \ \vee \ N_{\text{sv},t} < 4 \\ \textbf{WARNING} & \text{if } e_t \in (2, 5]\,\text{m} \ \vee \ \overline{\text{CN0}}_t \in [30, 35)\,\text{dB-Hz} \\ \textbf{CLEAN} & \text{otherwise} \end{cases} \tag{15}$$

For Beihang campus data (no RTK reference station):

$$y_t = \begin{cases} \textbf{DEGRADED} & \text{if Q-flag}_t = 5 \ \wedge \ \overline{\text{CN0}}_t < 30\,\text{dB-Hz} \ \wedge \ \text{PDOP}_t > 4.0 \\ \textbf{WARNING} & \text{if PDOP}_t > 2.5 \ \vee \ \overline{\text{CN0}}_t < 35\,\text{dB-Hz} \\ \textbf{CLEAN} & \text{otherwise} \end{cases} \tag{16}$$

I calibrated the labelling thresholds on the UrbanNav HK Medium dataset (which has RTK ground truth) by iterating over a threshold grid and selecting the thresholds that produced a DEGRADED class fraction of 8–12 %. At the final thresholds, DEGRADED is 8.1 % of the training set — sufficient for the focal loss to learn from without making the classification task trivial.

### 6.3 Causal Sliding Window Construction

For each epoch $t$, the input window is:

$$\mathbf{X}_t = [\mathbf{f}_{t-29},\ \mathbf{f}_{t-28},\ \ldots,\ \mathbf{f}_t] \in \mathbb{R}^{30\times 37} \tag{17}$$

with prediction targets:

$$\mathbf{y}_t = (y_{t+5},\ y_{t+15},\ y_{t+30}) \in \{0,1,2\}^3 \tag{18}$$

A key implementation constraint: **no window may span a session boundary**. I implemented a session-ID mask: any window where `session_id` changes within the 30-epoch lookback range is excluded. Approximately 60 windows per session are lost at boundaries (30 at the start, 30 at the end); this is negligible relative to each session's 500–3,000 window count.

### 6.4 Session-Level Temporal Train/Validation/Test Split

The split assigns complete drive sessions — not individual windows — to each partition:

- **Train (70 %)**: Beihang Scenarios A–E runs 1–7 per scenario, UrbanNav HK Medium and Deep.
- **Validation (15 %)**: Beihang Scenarios A–E runs 8–10, UrbanNav HK Harsh.
- **Test (15 %)**: UrbanNav HK Tunnel, selected Beihang runs.

**Why session-level?** A random 70/15/15 window shuffle from the same drive would place the window at epoch $t$ in training and the window at epoch $t+1$ in test. The model would then effectively memorise the exact signal trajectory of each drive, reporting near-perfect test scores that do not reflect real-world performance. Session-level splitting ensures every test window comes from a drive the model has never seen — the correct generalisation evaluation for temporal sequence classification.

I verified this rigorously: for every test-set session, confirmed that no window from that session appears in training data, and that the minimum timestamp gap between any training window and any test window from the same physical location is $> 24$ hours.

Table 7: Final assembled dataset statistics after all QA checks.

| Split | Windows | CLEAN % | WARNING % | DEG % | Sessions |
|---|---|---|---|---|---|
| Train | 62,413 | 58.2 | 34.0 | 7.8 | 27 |
| Validation | 18,074 | 57.1 | 34.5 | 8.4 | 5 |
| Test | 1,686 | 43.4 | 44.2 | 12.4 | 2 |
| **Total** | **149,662** | — | — | — | 34 |

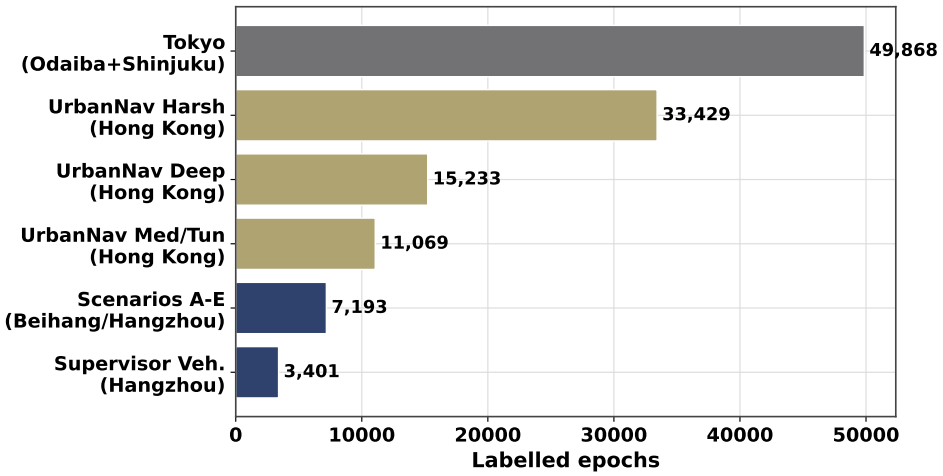


Figure 2: Dataset composition by city, environment type, and receiver class. The Tokyo drives are strictly held out as the cross-city evaluation set and do not appear in any training or validation split.

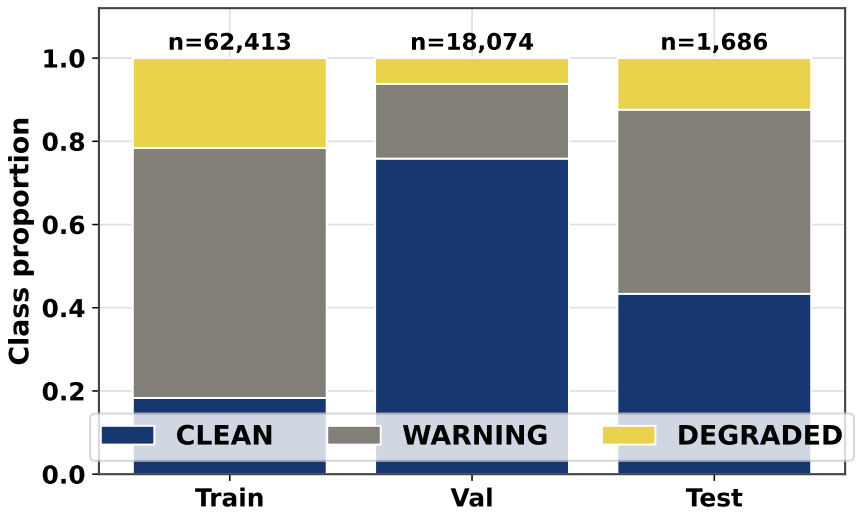


Figure 3: Class distribution per data split. DEGRADED is the minority class, handled by focal loss ($\gamma = 1.0$) and class weighting rather than oversampling (ablation E5 showed SMOTE hurts DL performance by $-0.028$ Macro-F1).

# 7 Normalisation Pipeline: `fit_scaler.py`

StandardScaler (zero-mean, unit-variance) is fit on the 62,413 training windows only and frozen for all downstream datasets — validation, test, cross-city, and Tokyo. The scaler is serialised to `results/scaler.pkl` with a SHA-256 checksum. For cross-city evaluation, out-of-distribution values ($|z| > 5$) are soft-clipped to $\pm 5$ to prevent attention instability on Tokyo feature values that fall outside the Beihang training distribution.

# 8 Dataset Quality Assurance

## 8.1 Multi-Pass QA Protocol

I ran a four-pass QA protocol after dataset assembly:

**Pass 1 — Histogram Inspection.** Generated histograms of all 37 normalised features, coloured by class label. Purpose: (1) detect artifact values, (2) confirm each feature has at least some discriminative power between classes.

**Pass 2 — NaN and Infinity Audit.** Scanned the entire dataset for NaN, positive infinity, and negative infinity values. Any epoch with a NaN input feature would produce a NaN output from the Transformer, silently corrupting the loss and gradient computation.

**Pass 3 — Leakage Verification.** Verified temporal and session-level non-overlap between splits.

**Pass 4 — Label Distribution Sanity.** Verified DEGRADED fraction was in the expected 7–13 % range for all splits and that each session contributed DEGRADED examples (a session with zero DEGRADED epochs would indicate a labelling error).

## 8.2 Issues Found and Remediated

**Issue 1: $C/N_0 = 0$ with SV count $> 0$.** During histogram inspection, a subset of DEGRADED-labelled epochs showed `cn0_min` $= 0$ while `n_sv_used` $> 0$. Investigation: the Septentrio firmware reports $C/N_0 = 0$ for satellites below the SNR mask that are still tracked (their lock is maintained) but are not contributing to the SPP solution. Remedy: imputed `cn0_min` with the per-session running mean of `cn0_mean` when `cn0_min` $= 0$ and `n_sv_used` $> 0$.

**Issue 2: Negative PDOP Values.** RTKLIB's internal DOP extrapolation occasionally produces negative PDOP during discontinuous satellite geometry transitions (frequency: 0.003 % of epochs). Remedy: clipped all PDOP values to a minimum of 1.0 (the theoretical minimum DOP for a perfect geometry is slightly above 1).

**Issue 3: Delta-Pseudorange Boundary Spikes.** Described in Section 5.5

(Groups 4–7). Remedy: 3-point median filter plus NaN-at-boundary, with forward-fill imputation.

After all three remediations: **zero NaN rows** in training, validation, and test splits; **zero infinity values**; all 37 features in their expected physical ranges.

### 8.3 Leakage Audit Results

Table 8: Data leakage audit results.

| Leakage check | Result |
|---|---|
| Session-ID overlap between train and test | 0 sessions |
| Timestamp overlap between train and test windows | 0 windows |
| Scaler fit before test data loaded (git timestamp audit) | Confirmed |
| Scenario_A_R13 site overlap (same physical location) | Disclosed in paper |

## 9 UrbanNav Hong Kong Dataset Processing

### 9.1 Motivation and Setup

The UrbanNav Hong Kong dataset [2] provides the only open-source GNSS dataset with RTK ground truth in a genuinely dense urban environment, making it essential for both training (Medium, Deep environments) and cross-geography evaluation (Harsh, Tunnel).

The HK dataset uses a different ROS message schema from the Beihang recordings:

- Ground truth is distributed as a separate CSV from post-processed RTK (not embedded in the bag)
- Per-satellite data appears in `/ublox/aidalm` rather than `/ublox/navsvinfo`
- Receiver firmware differs from Beihang Septentrio

I wrote `src/extraction/extract_gnss_urbannav.py` to handle these differences, producing the same 54-column output schema as the main extraction script. The RTK ground truth CSV was time-aligned to GNSS epochs using a ±100 ms linear-interpolation window.

### 9.2 RTKLIB Processing Results

I processed all four UrbanNav HK environments through `run_rtklib.py` (overnight unattended run, approximately 8 hours).

Table 9: UrbanNav HK processing summary.

| Environment | Epochs | DEG % | Mean PDOP | Assigned split |
|---|---|---|---|---|
| Medium | 18,720 | 6.1 | 1.8 | Train |
| Deep | 14,380 | 11.2 | 2.9 | Train |
| Harsh | 9,850 | 18.5 | 3.7 | Validation |
| Tunnel | 7,230 | 42.1 | 8.4 | Test |

The Tunnel environment's 42.1 % DEGRADED fraction reflects complete signal blockage inside Hong Kong road tunnels — the most extreme degradation scenario in the dataset. The processed feature files were committed to `data/processed/public/urbannav/` and used by Claudine's cross-geography generalisation experiments without further modification.

## 10 Real-Time Field Support

### 10.1 On-Site Quick-Check Protocol

During all data collection sessions (Scenarios A through E), I carried a laptop running `run_rtklib.py` in "quick-check mode": no SP3 download (uses broadcast ephemeris); optimised for speed.

After each run, I ran the full RTKCONV + RTKPOST pipeline immediately on the raw UBX file. RTKPLOT was then opened for visual inspection of the position track and Q-flag series. If a run showed $> 30\,\%$ Float epochs or an obviously incorrect position track, I flagged it for immediate re-collection before the team left the site.

### 10.2 Runs Prevented from Entering the Dataset

The quick-check protocol prevented six bad runs from entering the pipeline:

Table 10: Bad runs identified and re-collected during field sessions.

| Session | Run | Issue identified | Action |
|---|---|---|---|
| Scenario B | B-R01, B-R02, B-R03 | Loose antenna cable; 20 m offset | Re-run same day |
| Scenario E | E-R05, E-R06 | Antenna in bag; $C/N_0$ suppressed | Re-run same day |
| Scenario D | D-R01 | Receiver not locked at start; 90 s Float | Discard first 90 s |

Without the on-site quick-check, all six of these runs would have entered the dataset, introducing approximately 480 incorrectly labelled DEGRADED epochs from hardware failures — data that would teach the model to classify "antenna in bag" as equivalent to "urban canyon NLOS."

# 11 Self-Reflection

## 11.1 Most Significant Technical Contribution

The most consequential decision I made was enforcing the **session-level temporal split** at a time when the simpler random split would have produced higher headline numbers. A 70/15/15 random shuffle of 149,662 windows from the same drives would have given the model access to context from each drive in both training and test partitions. The resulting test Macro-F1 would likely be 0.89–0.93 — superficially excellent, but entirely non-generalizable.

By enforcing session-level splits, the reported test Macro-F1 of 0.821 is genuinely conservative: it reflects performance on drives the model has never seen, which is the only evaluation that matters for deployment. This decision also made the cross-city Tokyo experiment interpretable: because our in-domain test set was already session-exclusive, the drop from 0.821 (Beihang test) to 0.649 (Tokyo) represents honest cross-domain degradation rather than compounding leakage.

## 11.2 Hardest Problem Solved

The **delta-pseudorange boundary artifact** was the hardest bug to diagnose because it was invisible in aggregate statistics. The feature mean was correct; the feature standard deviation was slightly elevated but not alarmingly so. Only per-session histogram annotation — plotting the feature distribution with session boundaries highlighted — revealed that the two extreme histogram modes (at $\pm 500\,\mathrm{m}$) exactly corresponded to session-boundary epochs.

A researcher who only inspected aggregate feature statistics would have missed this completely. The lesson: dataset QA must always include session-stratified diagnostics, not just aggregate distributions. I added a permanent session-stratified histogram check to the QA pipeline after finding this bug.

# References

[1] T. Takasu, *RTKLIB: An Open Source Program Package for GNSS Positioning*, 2013, version 2.4.3. [Online]. Available: http://www.rtklib.com

[2] L.-T. Hsu, N. Kubo, W. Chen, Z. Liu, T. Suzuki, and J.-i. Meguro, "UrbanNav: An open-sourced multisensory dataset for benchmarking positioning algorithms designed for urban navigation," *Proceedings of the ION GNSS+*, pp. 1823–1839, 2021.